
Experiment 5

Power System State Estimation in MATLAB

Lab Report

Student Name HAMZA AHMAD
Registration Number 2022-EE-165
Section B
Power System Operation and Control Laboratory
Fall 2025

MATLAB code with comments

<https://github.com/hamza-2274/PSOC/tree/main/simplestateestimation>

```
function SE_Complete()  
  
    % === MAIN SCRIPT: SELab.m ===  
  
    clear  
  
    clc  
  
    tic  
  
    % Initial State Guesses  
  
    V = [1.06; 1.045; 1.01; 1.01772371530645; 1.01956929175548; 1.07;  
1.06158474200345; ...
```

```

1.09; 1.05599579866432; 1.05104189972113; 1.05694910281384;
1.05520754822189; ...

1.05040466613156; 1.03557823098035]; % [cite: 1]

delta = zeros(14,1); % [cite: 1]

% Line Data for Main Loop

B_cap = [0.0528 0.0492 0.0438 0.0340 0.0346 0.0128 0 0 0 0 0 0 0 0 0 0
0 0 0]'; % [cite: 2]

X = [0.05917; 0.22304; 0.19797; 0.17632; 0.17388; 0.17103; 0.04211;
0.1989; 0.25581; 0.13027; ...

0.17615; 0.11001; 0.0845; 0.27038; 0.19207; 0.19988; 0.34802;
0.20912; 0.55618; 0.20912]; % [cite: 3, 4]

R = [0.01938; 0.05403; 0.04699; 0.05811; 0.05695; 0.06701; 0.01335;
0.09498; 0.12291; 0.06615; ...

0; 0; 0.03181; 0.12711; 0.08205; 0.22092; 0.17093; 0; 0; 0]; %
[cite: 4, 5]

from_bus = [1; 1; 2; 2; 2; 3; 4; 4; 4; 5; 6; 6; 6; 7; 7; 9; 9; 10; 12;
13]; % [cite: 5, 6]

to_bus = [2; 5; 3; 4; 5; 4; 5; 7; 9; 6; 11; 12; 13; 8; 9; 10; 14; 11;
13; 14]; % [cite: 6, 7]

Z = R + 1j*X;

y = 1 ./ Z; % [cite: 8]

Nbus = length(V);

x = [delta(2:end); V]; % State vector initialization

% Get Admittance Matrix

[Ybus_matrix] = Ybuss();

% Generate Measurements with Noise

[b1] = SE_Measurments(); %

r = 0 + 0.02 .* randn(122, 1); % Add noise

b = b1 + r;

W = eye(122) / (0.02^2); % Weight matrix

% Iteration Parameters

```

```

max_iter = 70;

tolerance = 1e-4; % [cite: 10]

iter = 0;

converged = false;

fprintf('Iteration      Mismatch\n');
fprintf('-----\n');

while iter < max_iter && ~converged
    iter = iter + 1;

    % Calculate Jacobian and Function Mismatch

    [H] = State_Estimation_jacobian(V, delta, B_cap, from_bus, to_bus,
Ybus_matrix); % [cite: 11]

    [f] = SEFunc(V, delta, y, from_bus, to_bus, Ybus_matrix); % [cite:
11]

    e = b - f;

    % Gain Matrix and State Update

    G_matrix = (H' * W * H) \ (H' * W * e); % [cite: 12]

    x = x + G_matrix;

    % Update V and delta for next iteration

    V = x(Nbus:end);

    delta = [0; x(1:Nbus-1)];

    % Check Convergence

    max_mismatch = max(abs(G_matrix)); % [cite: 13]

    fprintf('%4d      %12.6f\n', iter, max_mismatch);

    if max_mismatch < tolerance
        converged = true;

        break %
    end
end

```

```

end

delta_degrees = rad2deg(x(1:Nbus-1));

disp('-----');
disp('Final Estimated Angles (Degrees, excluding slack):');
disp(delta_degrees);
disp('Final Estimated Voltages (p.u.):');
disp(V);

toc

end

% === SUBFUNCTION: Ybuss (Reconstructed) ===

function [Ybus] = Ybuss()

    % Data reconstructed from SE_Measurments vectors

    R = [0.01938; 0.05403; 0.04699; 0.05811; 0.05695; 0.06701; 0.01335;
0.09498; 0.12291; 0.06615; 0; 0; 0.03181; 0.12711; 0.08205; 0.22092;
0.17093; 0; 0; 0];

    X = [0.05917; 0.22304; 0.19797; 0.17632; 0.17388; 0.17103; 0.04211;
0.1989; 0.25581; 0.13027; 0.17615; 0.11001; 0.0845; 0.27038; 0.19207;
0.19988; 0.34802; 0.20912; 0.55618; 0.20912];

    B_sh = [0.0528 0.0492 0.0438 0.0340 0.0346 0.0128 0 0 0 0 0 0 0 0 0
0 0 0]';

    from_bus = [1; 1; 2; 2; 2; 3; 4; 4; 4; 5; 6; 6; 6; 7; 7; 9; 9; 10; 12;
13];

    to_bus = [2; 5; 3; 4; 5; 4; 5; 7; 9; 6; 11; 12; 13; 8; 9; 10; 14; 11;
13; 14];

    nbus = 14;

    nbr = length(from_bus);

    Ybus = zeros(nbus, nbus);

    Z = R + 1j*X;

    y = 1./Z;

    b_sh = 1j * B_sh;

    for k = 1:nbr

        i = from_bus(k);

        j = to_bus(k);

```

```

        Ybus(i,i) = Ybus(i,i) + y(k) + b_sh(k)/2;

        Ybus(j,j) = Ybus(j,j) + y(k) + b_sh(k)/2;

        Ybus(i,j) = Ybus(i,j) - y(k);

        Ybus(j,i) = Ybus(j,i) - y(k);

    end

end

% === SUBFUNCTION: SE_Measurments ===

function [b] = SE_Measurments()

    % [cite: 16] Function Header

    V = [1.06; 1.045; 1.01; 1.01772371530645; 1.01956929175548; 1.07;
1.06158474200345; 1.09; 1.05599579866432; 1.05104189972113;
1.05694910281384; 1.05520754822189; 1.05040466613156; 1.03557823098035];

    delta = zeros(14,1); % [cite: 17]

    X = [0.05917; 0.22304; 0.19797; 0.17632; 0.17388; 0.17103; 0.04211;
0.1989; 0.25581; 0.13027; 0.17615; 0.11001; 0.0845; 0.27038; 0.19207;
0.19988; 0.34802; 0.20912; 0.55618; 0.20912];

    R = [0.01938; 0.05403; 0.04699; 0.05811; 0.05695; 0.06701; 0.01335;
0.09498; 0.12291; 0.06615; 0; 0; 0.03181; 0.12711; 0.08205; 0.22092;
0.17093; 0; 0; 0];

    B = [0.0528 0.0492 0.0438 0.0340 0.0346 0.0128 0 0 0 0 0 0 0 0 0 0 0 0
0]';

    from_bus = [1; 1; 2; 2; 2; 3; 4; 4; 4; 5; 6; 6; 6; 7; 7; 9; 9; 10; 12;
13];

    to_bus = [2; 5; 3; 4; 5; 4; 5; 7; 9; 6; 11; 12; 13; 8; 9; 10; 14; 11;
13; 14]; % [cite: 20-22]

    Z = R + 1j*X;

    y = 1 ./ Z; % [cite: 26]

    [Ybus] = Ybuss();

    Ymag = abs(Ybus);

    Yang = angle(Ybus);

    branches = length(from_bus);

    Nbus = length(V);

    % Calculate True Power Flows for Measurements (Load Flow Logic
simplified)

```

```

P = zeros(Nbus,1);

Q = zeros(Nbus,1);

for i = 1:Nbus
    for j = 1:Nbus
        P(i) = P(i) + V(i)*V(j)*Ymag(i,j)*cos(Yang(i,j) - delta(i) +
delta(j)); % [cite: 33]
        Q(i) = Q(i) - V(i)*V(j)*Ymag(i,j)*sin(Yang(i,j) - delta(i) +
delta(j)); % [cite: 35]
    end
end

V_complex = V .* exp(1j * delta);

S = zeros(branches,1);

S1 = zeros(branches,1);

for a=1:branches
    i = from_bus(a);
    j = to_bus(a);

    S(a) = V_complex(i)*conj(y(a)*(V_complex(i)-V_complex(j))); % [cite:
66]
    S1(a) = V_complex(j)*conj(y(a)*(V_complex(j)-V_complex(i)));

end

P1 = real(S); Q1 = imag(S);

P2 = real(S1); Q2 = imag(S1);


b = zeros(3*Nbus+4*branches,1);

b(1:Nbus) = V;

b(Nbus+1:2*Nbus) = P;

b(2*Nbus+1:3*Nbus) = Q;

b(3*Nbus+1:3*Nbus+branches) = P1;

b(3*Nbus+branches+1:3*Nbus+2*branches) = P2;

b(3*Nbus+2*branches+1:3*Nbus+3*branches) = Q1;

b(3*Nbus+3*branches+1:3*Nbus+4*branches) = Q2; % [cite: 69]

end

% === SUBFUNCTION: SEFunc ===

function [f] = SEFunc(V, delta, y, from_bus, to_bus, Ybuss) % [cite: 70]

```

```

branches = length(from_bus);

Nbus = length(V);

Ymag = abs(Ybuss);

Yang = angle(Ybuss);

f = zeros(3*Nbus+4*branches,1);

Q = zeros(Nbus,1);

P = zeros(Nbus,1);

for i = 1:Nbus
    for j = 1:Nbus
        P(i) = P(i) + V(i)*V(j)*Ymag(i,j)*cos(Yang(i,j) - delta(i) +
delta(j)); % [cite: 71]
        Q(i) = Q(i) - V(i)*V(j)*Ymag(i,j)*sin(Yang(i,j) - delta(i) +
delta(j)); % [cite: 72]
    end
end

V_complex = V .* exp(1j * delta);

S = zeros(branches,1);

S1 = zeros(branches,1);

for a=1:branches
    i = from_bus(a);
    j = to_bus(a);
    S(a) = V_complex(i)*conj(Y(a)*(V_complex(i)-V_complex(j))); % [cite:
73]
    S1(a) = V_complex(j)*conj(Y(a)*(V_complex(j)-V_complex(i)));
end

P1 = real(S); Q1 = imag(S);

P2 = real(S1); Q2 = imag(S1);

f(1:Nbus) = V;

f(Nbus+1:2*Nbus) = P;

f(2*Nbus+1:3*Nbus) = Q;

f(3*Nbus+1:3*Nbus+branches) = P1;

```

```

f(3*Nbus+branches+1:3*Nbus+2*branches) = P2;

f(3*Nbus+2*branches+1:3*Nbus+3*branches) = Q1;

f(3*Nbus+3*branches+1:3*Nbus+4*branches) = Q2; % [cite: 74]

end

% === SUBFUNCTION: State_Estimation_jacobian ===

function [J] = State_Estimation_jacobian(V, delta, B, from_bus, to_bus,
Ybuss) % [cite: 75]

    branches = length(from_bus);

    Nbus = length(V);

    Ymag = abs(Ybuss);

    Yang = angle(Ybuss);

    % J1: dV/d(delta) = 0

    J1 = zeros(Nbus, Nbus-1);

    % J2: dV/dV = Identity

    J2 = eye(Nbus);

    % J3: dP/d(delta) [cite: 80]

    J3 = zeros(Nbus, Nbus-1);

    for i=1:Nbus

        for j=2:Nbus

            if i~=j

                J3(i,j-1) =
-V(i)*V(j)*Ymag(i,j)*sin(Yang(i,j)+delta(j)-delta(i));

            else

                sum_term = 0;

                for k = 1:Nbus

                    if k ~= i

                        sum_term = sum_term +
V(i)*V(k)*Ymag(i,k)*sin(Yang(i,k) + delta(k) - delta(i));

                    end

                end

                J3(i,j-1) = sum_term;

            end

        end

    end

end

```



```

% J4: dP/dV [cite: 85]

J4 = zeros(Nbus,Nbus);

for i=1:Nbus
    for j=1:Nbus
        if i~=j
            J4(i,j) = V(i)*Ymag(i,j)*cos(Yang(i,j)+delta(j)-delta(i));
        else
            diagonal_term = 2 * V(i) * Ymag(i,i) * cos(Yang(i,i));
            sum_term = 0;
            for k = 1:Nbus
                if k ~= i
                    sum_term = sum_term + V(k)*Ymag(i,k)*cos(Yang(i,k) +
delta(k)- delta(i));
                end
            end
            J4(i,j) = diagonal_term + sum_term;
        end
    end
end

% J5: dQ/d(delta) [cite: 91]

J5 = zeros(Nbus,Nbus-1);

for i=1:Nbus
    for j=2:Nbus
        if i~=j
            J5(i,j-1) =
-V(i)*V(j)*Ymag(i,j)*cos(Yang(i,j)+delta(j)-delta(i));
        else
            sum_term = 0;
            for k = 1:Nbus
                if k ~= i
                    sum_term = sum_term +
V(i)*V(k)*Ymag(i,k)*cos(Yang(i,k) + delta(k) - delta(i));
                end
            end
        end
    end
end

```

```

        J5(i,j-1) = sum_term;
    end
end
end

% J6: dQ/dV [cite: 96]
J6 = zeros(Nbus,Nbus);
for i=1:Nbus
    for j=1:Nbus
        if i~=j
            J6(i,j) = -V(i)*Ymag(i,j)*sin(Yang(i,j)+delta(j)-delta(i));
        else
            diagonal_term = -2 * V(i)* Ymag(i,i) * sin(Yang(i,i));
            sum_term = 0;
            for k = 1:Nbus
                if k ~= i
                    sum_term = sum_term - V(k)*Ymag(i,k)*sin(Yang(i,k) +
delta(k) - delta(i) );
                end
            end
            J6(i,j) = diagonal_term + sum_term;
        end
    end
end

% J7: dPij/d(delta) [cite: 102]
J7 = zeros(branches,Nbus-1);
for a=1:length(from_bus)
    i = from_bus(a); j = to_bus(a);
    if i~=1, J7(a,i-1) =
V(i)*V(j)*Ymag(i,j)*sin(Yang(i,j)+delta(j)-delta(i)); end
    if j~=1, J7(a,j-1) =
-V(i)*V(j)*Ymag(i,j)*sin(Yang(i,j)+delta(j)-delta(i)); end
end

% J8: dPij/dV [cite: 104]
J8 = zeros(branches,Nbus);

```

```

for a=1:length(from_bus)

    i = from_bus(a); j = to_bus(a);

    J8(a,i) = (-2*V(i)*real(Ybuss(i,j))) +
(V(j)*Ymag(i,j)*cos(Yang(i,j)+delta(j)-delta(i)));

    J8(a,j) = V(i)*Ymag(i,j)*cos(Yang(i,j)+delta(j)-delta(i));

end

% J9: dPji/d(delta) [cite: 105]

J9 = zeros(branches,Nbus-1);

for a=1:length(from_bus)

    i = from_bus(a); j = to_bus(a);

    if i~=1, J9(a,i-1) =
-V(i)*V(j)*Ymag(j,i)*sin(Yang(i,j)+delta(i)-delta(j)); end

    if j~=1, J9(a,j-1) =
V(i)*V(j)*Ymag(j,i)*sin(Yang(i,j)+delta(i)-delta(j)); end

end

% J10: dPji/dV [cite: 106]

J10 = zeros(branches,Nbus);

for a=1:length(from_bus)

    i = from_bus(a); j = to_bus(a);

    J10(a,i) = V(j)*Ymag(j,i)*cos(Yang(i,j)+delta(i)-delta(j));

    J10(a,j) = (-2*V(j)*Ymag(j,i)*cos(Yang(j,i))) +
(V(j)*Ymag(j,i)*cos(Yang(i,j)+delta(i)-delta(j)));

end

% J11: dQij/d(delta) [cite: 107]

J11 = zeros(branches,Nbus-1);

for a=1:length(from_bus)

    i = from_bus(a); j = to_bus(a);

    if i~=1, J11(a,i-1) =
V(i)*V(j)*Ymag(j,i)*cos(Yang(i,j)+delta(j)-delta(i)); end

    if j~=1, J11(a,j-1) =
-V(i)*V(j)*Ymag(j,i)*cos(Yang(i,j)+delta(j)-delta(i)); end

end

% J12: dQij/dV [cite: 110]

J12 = zeros(branches,Nbus);

for a=1:length(from_bus)

```

```

i = from_bus(a); j = to_bus(a);

J12(a,j) = -V(i)*Ymag(j,i)*sin(Yang(j,i)+delta(j)-delta(i));

BB = (B(a)/2) - (imag(Ybuss(i,j)));

J12(a,i) = (-2*V(i)*BB) -
(V(j)*Ymag(j,i)*sin(Yang(j,i)+delta(j)-delta(i)));

end

% J13: dQji/d(delta) [cite: 111]

J13 = zeros(branches,Nbus-1);

for a=1:length(from_bus)

    i = from_bus(a); j = to_bus(a);

    if i~=1, J13(a,i-1) =
-V(i)*V(j)*Ymag(j,i)*cos(Yang(j,i)+delta(i)-delta(j)); end

    if j~=1, J13(a,j-1) =
V(i)*V(j)*Ymag(j,i)*cos(Yang(j,i)+delta(i)-delta(j)); end

end

% J14: dQji/dV [cite: 114]

J14 = zeros(branches,Nbus);

for a=1:length(from_bus)

    i = from_bus(a); j = to_bus(a);

    J14(a,i) = -V(j)*Ymag(j,i)*sin(Yang(j,i)+delta(i)-delta(j));

    BB = (B(a)/2) - (imag(Ybuss(i,j)));

    J14(a,j) = (-2*V(j)*BB) -
(V(i)*Ymag(j,i)*sin(Yang(j,i)+delta(i)-delta(j)));

end

J = [J1 J2; J3 J4; J5 J6; J7 J8; J9 J10; J11 J12; J13 J14]; %
[cite: 115]

end

```

RESULTS

Measurement set and noise model?

The measurement vector **b** consists of a total of **122 elements**, constructed from different types of power system measurements. The structure of the measurement vector is organized as follows:

1. **Voltage Magnitudes (V):**
14 measurements corresponding to the voltage magnitudes at each bus.
2. **Active Power Injections (P):**
14 measurements representing the active power injections at each bus.
3. **Reactive Power Injections (Q):**
14 measurements representing the reactive power injections at each bus.
4. **Branch Active Power Flows ($P_{(i,j)}$ and $P_{(j,i)}$):**
For 20 transmission branches, active power flow is measured in both directions.
This results in $20 \times 2 = 40$ measurements.
5. **Branch Reactive Power Flows ($Q_{(i,j)}$ and $Q_{(j,i)}$):**
For the same 20 branches, reactive power flow is measured in both directions.
This results in $20 \times 2 = 40$ measurements.

Adding all components together:

$$14 (V) + 14 (P) + 14 (Q) + 40 (P_{\text{flow}}) + 40 (Q_{\text{flow}}) = 122 \text{ measurements}$$

The measurement vector is generated by adding Gaussian noise to the true (noise-free) measurement vector **b₁**. The noise model assumes zero-mean Gaussian noise with a standard deviation of 0.02. This relationship is expressed as:

$$\mathbf{b} = \mathbf{b}_1 + \mathbf{r}$$

where

$$\mathbf{r} = 0.02 \times \text{randn}(122, 1)$$

The **weighting matrix W** used in the Weighted Least Squares (WLS) state estimation is defined as a diagonal matrix with identical variances for all measurements:

$$\mathbf{W} = \text{eye}(122) / (0.02^2)$$

This weighting matrix reflects the assumption that all measurements are independent and have equal noise variance.

Iteration convergence table?

Command Window

Iteration	Mismatch
-----------	----------

1	0.005277
---	----------

2	0.000037
---	----------

Final Estimated Angles (Degrees, excluding slack):

-0.0131

-0.0627

0.0335

0.0186

0.1824

0.2192

0.1128

0.0945

0.1486

0.1991

0.2869

0.2077

0.3021

Final estimated states (voltages, angles)?

Final Estimated Voltages (p.u.):

1.0588

1.0440

1.0095

1.0173

1.0185

1.0683

1.0623

1.0923

1.0568

1.0492

1.0539

1.0520

1.0485

1.0346

Elapsed time is 0.246467 seconds.

fx >>